Traffic Law and Case Retrieval System with RAG and KP-Case Alignment

314551046 Lin Guancheng, 111705068 Wang Ziyi,
314706037 Su Hongcheng, 314706028 Qiu Dingyue
National Yang Ming Chiao Tung University

1 Introduction

The project addresses the limitations of traditional Retrieval-Augmented Generation (RAG) systems when applied to the legal domain. Traditional systems suffer from insufficient knowledge retrieval, often finding facts without providing application guidance. Furthermore, they exhibit limited reasoning capabilities, failing to effectively apply retrieved knowledge to solve practical problems. Context fragmentation is another issue, where the correlation between statutes and legal precedents is difficult to capture.

The specific Information Retrieval (IR) tasks addressed include vector retrieval, bi-directional expansion retrieval (KP-Case Alignment), complete document assembly, and deduplication/merging of results. The proposed system, built as a FastAPI service, aims to provide intelligent retrieval of traffic regulations and precedents through semantic vector search and multi-stage reranking.

2 Literature Review

The project references two key methodologies: RAG+ and LightRAG.

RAG+ [2]: This approach emphasizes the construction of a "Knowledge + Application" system. It utilizes Bayes' Theorem to model the relationship between knowledge points and their applications. The framework involves calculating posterior probabilities to infer the probability of an event (Application) given the evidence (Knowledge).

LightRAG [3]: This framework employs a graph-based text indexing system utilizing a dual-level retrieval paradigm. It extracts entities and relationships to build an index graph, facilitating the retrieval of both specific entities and broader contexts.

3 Dataset

The project utilizes a dual-source dataset comprising laws and legal precedents. To rigorously evaluate the system, we generated two distinct types of queries.

3.1 Data Sources

- **Statutes (Knowledge)**: The dataset includes 93 regulations from the "Road Traffic Management and Penalty Act," sourced from the National Laws Database in CSV format.
- **Precedents (Application)**: The dataset includes traffic-related judgments sourced from the Judicial Yuan Open Data Platform. The vector database was populated using judgments from **August 2025**.

3.2 Query Generation

1. Synthetic Law-based Queries: We programmatically generated questions to test direct legal knowledge retrieval.

- **Method**: We iterated through all laws and utilized regular expressions to extract specific items (e.g., "一、", "二、"). For each item, a penalty-type question was generated (e.g., "What is the penalty for...?").
- **Ground Truth**: The specific law text corresponding to the generated question.

2. Case-based Queries: To simulate real-world usage, we utilized case descriptions as queries.

- **Method**: We extracted case chunks from judgments dated **September 2025** (ensuring temporal separation from the August training data). The raw chunk text served as the query.
- **Ground Truth**: The specific law referenced in the judgment.

4 Baseline

We established a standard baseline to evaluate the effectiveness of our proposed method. The baseline retrieval process operates as follows:

1. **Vector Search**: The input query is embedded and used to perform a cosine similarity search against the statute collection in the vector database.
2. **Top-K Retrieval**: The system retrieves the top K most similar law chunks.
3. **Answer Extraction**: For each retrieved chunk, the content of the `cited_law` field is extracted as the predicted answer.
4. **Evaluation**: These predictions are compared against the ground truth to compute standard IR metrics.

5 Main Approach

The proposed solution is a hybrid retrieval system featuring **KP-Case Alignment** and **Multi-stage Reranking**.

5.1 System Architecture

The core processing pipeline includes the following modules:

- **Vector Search** (`vector_search.py`): Utilizes ChromaDB to store and retrieve embeddings.
- **KP-Case Alignment** (`alignment.py`): Maps abstract legal Knowledge Points (KP) to concrete Case Applications. This relationship is managed via an SQLite table `kp_app_mapping`.
- **Reranking** (`reranker.py`): Re-orders retrieved results to prioritize legal relevance and reduce noise.

6 Evaluation Metric

We employ standard Information Retrieval metrics: Precision@K, Mean Reciprocal Rank (MRR), and Hit@K. Our performance targets include a Hit@5 (Cases) ≥ 0.60 and an average latency ≤ 3 seconds.

7 Results

We compared the Baseline (direct law retrieval) against our Proposed Method (Law retrieval with Case-based expansion).

7.1 Quantitative Results

Scenario 1: Law-based Queries Table 1 shows the performance on synthetic questions derived directly from statutes.

Metric	@1	@3	@5	@10
Baseline				
Precision	0.41	0.21	0.14	0.08
MRR	0.41	0.51	0.53	0.54
Hit	**0.41**	**0.63**	**0.72**	**0.82**
Our Method (With Expansion)				
Precision	0.32	0.21	0.13	0.03
MRR	0.32	0.44	0.47	0.54
Hit	0.32	0.53	0.67	0.75

Table 1: Performance on Synthetic Law-based Queries.

Scenario 2: Case-based Queries Table 2 presents the results for queries based on real-world case descriptions (September data).

Metric	@1	@3	@5	@10
Baseline				
Precision	0.13	0.08	0.06	0.05
MRR	0.13	0.24	0.28	0.31
Hit	0.13	0.26	0.35	0.43
Our Method (With Expansion)				
Precision	**0.23**	**0.16**	**0.14**	**0.08**
MRR	**0.23**	**0.46**	**0.52**	**0.54**
Hit	**0.23**	**0.56**	**0.65**	**0.77**

Table 2: Performance on Case-based Queries (September Data).

8 Error Analysis

To better understand the properties of our system and the sources of error, we conducted two experiments analyzing the impact of query type and retrieval depth.

8.1 Experiment 1: Query Verbosity Sensitivity

We analyzed the differential performance of the expansion mechanism on short, factual queries versus long, narrative queries.

Observation: As illustrated in Table 3, the proposed method exhibits a dual nature. It provides a massive boost (+85.7% relative improvement in Hit@5) for Case-based queries but causes a regression (-6.9% relative drop in Hit@5) for Law-based queries.

Surprise: The degradation on synthetic data was unexpected. We hypothesized that more context

Dataset	Base Hit@5	Ours Hit@5	Delta
Synthetic (Laws)	0.72	0.67	-6.9%
Real (Cases)	0.35	0.65	**+85.7%**

Table 3: Impact of expansion on different query types.

would universally aid retrieval. However, for concise, fact-based queries, the expansion mechanism retrieved adjacent but legally distinct cases, introducing noise that diluted the correct statute's vector score.

Take-away: There is no "one-size-fits-all" retrieval strategy. The system requires a router to classify query intent: simple factual lookups should bypass the expansion layer, while narrative descriptions must use it.

8.2 Experiment 2: The "Single Truth" Penalty

We analyzed the Precision@K degradation as K increases. In standard IR, lower precision at high K is expected, but the drop in our system was particularly sharp (from 0.32 at @1 to 0.03 at @10 for Law queries).

Analysis: This error profile stems from the nature of the legal domain. Unlike general web search where multiple pages might be relevant, a specific traffic violation typically corresponds to exactly one statute.

- **Pros**: The system is highly decisive when it works (high MRR).
- **Cons**: Increasing K to improve Recall (Hit Rate) unavoidably destroys Precision because $K - 1$ retrieved items are mathematically guaranteed to be false positives.

8.3 Error Sources

The primary source of error identified is **Vector Database Noise**. The chunks stored in ChromaDB contain unprocessed case details (dates, names, procedural text) that are irrelevant to the legal principle. This noise aligns serendipitously with parts of the expansion queries, leading to "hallucinated" retrieval where the system retrieves a case that sounds similar but references a different law.

9 Future Work

Future work focuses on bridging the gap between current results and performance targets:

- **Data Cleaning**: Improving chunk quality to reduce noise in the stored data.
- **Query Routing**: Implementing a classifier to dynamically toggle expansion based on query length and complexity.
- **Enhanced Alignment**: Improving the `alignment_builder.py` logic to further boost Hit rates for case queries.

Code

`https://github.com/kclin111/IR-final-project`

Contribution of each member

- 314551046 林冠丞25%
- 111705068 王子儀25%
- 314706037 蘇泓丞25%
- 314706028 邱鼎越25%

References

[1] Project Slides & Documentation. 2025. Traffic Law and Case Retrieval System Architecture.

[2] RAG+: Enhancing Retrieval-Augmented Generation with Application-Aware Reasoning. 2025. *ACL Anthology*. `https://aclanthology.org/2025.emnlp-main.1630/`

[3] LightRAG: Simple and Fast Retrieval-Augmented Generation. 2025. *ACL Anthology*. `https://aclanthology.org/2025.findings-emnlp.568/`